

TECHNICAL ASSESSMENT

README

Setup, artifacts, and how to run the harness.

Arash Khajooei · Deriv · AI Engineer

A replayable, offline evaluation pipeline for comparing two (or more) prompting strategies for a retrieval-augmented support Q&A system. It retrieves evidence deterministically, scores answers with a mix of code-based checks and one controlled LLM judgment stage, detects safety and grounding failures, and produces a promotion recommendation with stated reasoning — not just a leaderboard.

Three documents, three jobs. [TASK.md](#) is the brief, reproduced in full with a requirement-by-requirement coverage map. This README covers setup and artifacts. [HANDBOOK.md](#) is the complete walkthrough — every design decision, with worked examples and the reasoning behind each. PDF versions of all three are in the repository root.

Status: complete. All ten build steps are implemented: deterministic retrieval, rule-based scoring, the one controlled LLM review stage (cache/live/stub), the failure taxonomy, safety-gated aggregation and recommendation, the explainability view, the run manifest, the `validate.py` consistency checker, and the test suite (86 tests).

Contents

- [Quickstart](#)
- [Dashboard \(optional UI\)](#)
- [Running with Docker](#)
- [Repository layout](#)
- [Artifacts reference](#) — what's in each generated file
- [How the recommendation is computed](#) — the aggregation rule, plain English
- [LLM review stage: enabling live judging](#)
- [Validation](#)
- [Design principles this repo follows](#)
- [Design tradeoffs and why](#)
- [Known limitations](#)
- [What I'd add next](#)
- [A note on the sample answer data](#)

Quickstart

```
uv venv --python 3.12 .venv          # or: python3 -m venv .venv
uv pip install --python .venv/bin/python -r requirements-dev.txt
.venv/bin/python -m pytest -q        # 86 tests
.venv/bin/python run.py              # regenerate every artifact
.venv/bin/python validate.py         # check the artifacts are complete and consistent
```

Or, with `make` :

```
make install
make test
make run
make validate
```

The harness targets **Python 3.9+** for compatibility with older system interpreters (verified with `py_compile` against the system 3.9 interpreter); it has been developed and tested on 3.12.

`run.py` regenerates every artifact: `retrieval.json`, `automated_scores.json`, `llm_review.json`, `llm_calls.jsonl`, `failure_taxonomy.json`, `recommendation.md`, `review_report.md`, and `run_manifest.json`. `validate.py` checks that a completed run's artifacts are present, well-formed, internally consistent, and that `recommendation.md` is reproducible from the stored artifacts alone — see "Validation" below for exactly what it checks.

Dashboard (optional UI)

A local web dashboard that shows every pipeline stage on one screen and lets you run the harness without a terminal. It exists because the pipeline emits eight artifact files, and cross-referencing them by hand to answer "why did this variant lose?" is slow.

It is not part of the graded harness — see [Scope and safety](#).

Quick start

From a clean checkout:

```
# 1. create a virtualenv (once)
uv venv --python 3.12 .venv          # or: python3 -m venv .venv

# 2. install the harness + dashboard dependencies (once)
uv pip install --python .venv/bin/python -r requirements-frontend.txt
#   (this file includes requirements.txt, so it covers both)

# 3. generate the artifacts the dashboard displays (once, or any time inputs change)
.venv/bin/python run.py

# 4. start the dashboard
.venv/bin/uvicorn frontend.server:app --host 127.0.0.1 --port 5050
```

Then open <http://127.0.0.1:5050>.

Alternative launch	<code>.venv/bin/python frontend/server.py</code> — equivalent, calls <code>uvicorn.run()</code> with the same settings
Live reload while editing the UI	add <code>--reload</code> to the uvicorn command
Different port	change <code>--port</code> ; nothing is hardcoded to 5050
Interactive API docs	http://127.0.0.1:5050/api/docs (FastAPI's generated Swagger UI)
Stop it	<code>Ctrl+C</code> , or <code>pkill -f "uvicorn frontend.server"</code>

If you skip step 3, the dashboard still loads but shows an empty state — click **Run Pipeline** in the header and it will generate everything for you.

Enabling the live LLM judge from the dashboard. The server passes its own environment to the subprocesses it launches, so export the key *before* starting it:

```
export OPENAI_API_KEY=...      # or ANTHROPIC_API_KEY, matching judge.provider
.venv/bin/uvicorn frontend.server:app --host 127.0.0.1 --port 5050
```

Now **Run Pipeline** will make a real judge call (once — subsequent runs replay from cache). Without a key it falls back to `cache` , then `stub` , and the Overview badge shows which one served the run.

What the dashboard shows

Everything comes from one `GET /api/state` call that returns the parsed contents of every artifact. Nothing is recomputed differently for display — the recommendation in particular is produced by importing the pipeline's own `compute_recommendation()` , so the dashboard cannot show a verdict the harness would not.

Header (always visible)

ELEMENT	DATA IT CARRIES
original sample / custom fixture badge	SHA-256 of the three current input files compared against a snapshot of the committed originals
Run Pipeline button	runs <code>python run.py</code> ; streams stdout/stderr + exit code into a console panel
Validate button	runs <code>python validate.py</code>
Run Tests button	runs <code>pytest</code>

Tab 1 — Overview

ELEMENT	SOURCE
Selected variant (large, green)	<code>recommendation.selected_variant</code>
N queries × M variants	counts from <code>queries.json</code> / <code>candidate_answers.json</code>
Judge badge — live / cache / stub + provider/model	<code>run_manifest.json</code> → <code>judge.backend_used</code> . Orange for <code>stub</code> , so a rule-derived result is never mistaken for real model judgment
"Why" bullet list	<code>recommendation.reasons</code> — the aggregation's own reasoning
Tradeoffs block (when present)	<code>recommendation.tradeoffs</code> — fires when a disqualified variant beat the winner on judged clarity
Collapsible raw files	full text of <code>recommendation.md</code> and <code>review_report.md</code>

Tab 2 — Per-Query

One card per question, carrying every stage's output for that question:

ELEMENT	SOURCE ARTIFACT
Query id + <code>risk_level</code> badge	<code>queries.json</code>
The user question	<code>queries.json</code>
Retrieved evidence — doc id, BM25 score, title, full text	<code>retrieval.json</code>
Green <code>expected</code> badge on a passage	that doc id appears in the query's <code>expected_doc_ids</code> — i.e. retrieval found the right evidence
Each variant's answer text	<code>candidate_answers.json</code>
Check pills — <code>retrieval hit</code> , <code>must-include</code> , no banned claim / BANNED CLAIM , <code>grounding N</code>	<code>automated_scores.json</code>
Failure tags	<code>failure_taxonomy.json</code>
<code>check details</code> expander	the <code>notes</code> field — <i>which</i> phrase matched via <i>which</i> matching rung (<code>exact</code> / <code>subsequence</code> / <code>stemmed_subsequence</code>)
LLM judge — winner: X, per-variant <code>faith</code> / <code>clarity</code> / <code>overclaim</code> , and the justification text	<code>llm_review.json</code>

This is the debugging view: when a variant loses, this tab shows why without opening a JSON file.

Tab 3 — Variants

The head-to-head comparison table. Columns are ordered to match the *decision* order, not score order:

COLUMN	SOURCE
SAFETY GATE — <code>passes</code> or <code>DISQUALIFIED</code> + the disqualifying queries	<code>recommendation.verdicts[].disqualified</code> / <code>.disqualifying_reasons</code>
COMPOSITE	<code>recommendation.verdicts[].composite_score</code>
JUDGE FAITHFULNESS / JUDGE CLARITY	mean across all queries, from <code>llm_review.json</code>
JUDGE WINS	how many queries the judge picked that variant for
FAILURE TAGS	tag counts aggregated from <code>failure_taxonomy.json</code>

A footnote states that the gate is a veto, so a reader cannot mistake the composite gap for the cause of the decision.

Tab 4 — Logs & Provenance

SECTION	DATA IT CARRIES
LLM call log — one row per attempt: backend, ok/failed, timestamp, model, truncated prompt hash, validation errors	every line of <code>llm_calls.jsonl</code> , including <i>failed</i> attempts
Raw <code>llm_calls.jsonl</code>	collapsible, full text
Run provenance — generated-at, git commit, config hash, judge model, judge backend used, Python version, and a SHA-256 per input file	<code>run_manifest.json</code>

The call log is where the caching design is visible: repeated runs on an unchanged fixture show one `live` row followed by `cache` rows sharing an identical prompt hash.

Tab 5 — Fixture

ELEMENT	DATA IT CARRIES
Current input counts — KB documents, queries, variants (listed by name)	read from the live input files, never hardcoded
Collapsible viewers for <code>kb.json</code> , <code>queries.json</code> , <code>candidate_answers.json</code>	full file contents
Three file pickers + Upload & replace	see below
Restore original sample	copies the committed originals back over the current inputs

Swapping fixtures from the browser

Upload replacement `kb.json` / `queries.json` / `candidate_answers.json`, and the server validates all three with the harness's own `load_inputs()` **before** overwriting anything. A fixture that fails schema or referential-integrity checks is rejected with the exact errors the pipeline would print, and the working fixture is left untouched:

```
success: False
ERROR: [ERROR] DANGLING_DOC_ID at queries.json[query_id=Q1].expected_doc_ids:
      expected_doc_ids references unknown doc_id 'D_NOPE'
```

Because it reuses the pipeline's validator rather than a second copy of the rules, a file the pipeline would reject is a file the upload rejects, by construction.

Once a valid fixture is uploaded, click **Run Pipeline** to evaluate it. This was verified end to end against a fixture with entirely different document ids, different query ids, and **three** variants (`baseline` / `concise_v2` / `risky_v3`): the pipeline correctly disqualified `risky_v3` for a banned claim on the high-risk query and promoted `baseline` on composite score — with no code changes.

The HTTP API

METHOD	PATH	PURPOSE
GET	<code>/</code>	the dashboard page
GET	<code>/api/state</code>	every artifact, parsed, as one JSON payload
POST	<code>/api/run</code>	run <code>run.py</code> , return stdout/stderr/exit code
POST	<code>/api/validate</code>	run <code>validate.py</code>
POST	<code>/api/test</code>	run <code>pytest</code>
POST	<code>/api/upload-fixture</code>	multipart upload of <code>kb</code> / <code>queries</code> / <code>answers</code> ; validated before write
POST	<code>/api/restore-sample</code>	restore the committed sample inputs

Troubleshooting

SYMPTOM	CAUSE / FIX
Address already in use	something else holds 5050 — use <code>--port 5051</code> , or <code>kill -f "uvicorn frontend.server"</code>
Dashboard loads but everything is empty	no artifacts yet — click Run Pipeline , or run <code>python run.py</code>
Judge badge shows orange <code>stub</code>	no API key and no cached call for this fixture. Export a key and restart the server, or accept the rule-derived fallback
Red banner with validation errors	the current input files are invalid — the exact errors are shown; fix them or click Restore original sample
<code>ModuleNotFoundError: fastapi</code>	install the frontend extras: <code>uv pip install -r requirements-frontend.txt</code>

Scope and safety

The dashboard is **not part of the graded harness**. Its dependencies live in `requirements-frontend.txt`, and `run.py`, `validate.py` and the test suite import nothing from `frontend/` — they behave identically whether or not it is ever started.

It binds to **loopback only** and runs without auto-reload by default, because it executes local subprocesses (`run.py`, `validate.py`, `pytest`) on request. Uploads are size-capped and validated before touching disk. It is a local developer tool and should not be exposed beyond localhost.

Running with Docker

A `Dockerfile` and `docker-compose.yml` are included so the harness and its dashboard can be run without installing Python or any dependencies on the host.

One command

```
docker compose up --build
```

Then open <http://127.0.0.1:5050>.

Or without compose:

```
docker build -t deriv-eval .
docker run --rm -p 127.0.0.1:5050:5050 deriv-eval
```

Running the pipeline, validator or tests instead

The image's default command starts the dashboard, but any of the harness's entrypoints can be run directly:

```
docker run --rm deriv-eval python run.py          # regenerate all artifacts
docker run --rm deriv-eval python validate.py      # check artifact consistency
docker run --rm deriv-eval python -m pytest -q     # the test suite
```

With compose:

```
docker compose run --rm harness python run.py
docker compose run --rm harness python validate.py
docker compose run --rm harness python -m pytest -q
```

What the image contains

Base	python:3.12-slim
Dependencies	harness + dashboard + dev + the optional LLM SDK, so live judging works if a key is supplied
Runs as	non-root (<code>appuser</code> , uid 10001)
Artifacts	generated at build time via <code>RUN python run.py</code> , so the dashboard has results the moment the container starts
Judge cache	<code>llm_calls.jsonl</code> is copied in, so the container reproduces the real recorded LLM verdicts offline, with no API key and no network

That last point is worth noting: a fresh `docker run` with no configuration at all shows genuine model judgments, not the deterministic stub, because the cached call travels with the image.

Enabling the live LLM judge

Pass a key through as an environment variable:

```
docker run --rm -p 127.0.0.1:5050:5050 \
  -e OPENAI_API_KEY="$OPENAI_API_KEY" \
  deriv-eval
```

Or uncomment the matching line under `environment:` in `docker-compose.yml` and run `docker compose up` .

Without a key the judge replays the baked-in cache, and falls back to the deterministic stub if the fixture has changed. The pipeline completes either way, and the Overview badge always shows which backend served the run.

Getting artifacts onto the host

The container is self-contained by default. To retrieve generated files:

```
docker compose cp harness:/app/recommendation.md .
```

Or use the `dev` service, which bind-mounts your working copy so the container runs your local code and writes artifacts straight back into the checkout:


```
docker compose --profile dev up dev      # dashboard on http://127.0.0.1:5051
```

A note on network binding

Inside the container the server binds `0.0.0.0`, which is required for Docker port publishing to reach the process at all — `127.0.0.1` inside a container is only reachable from inside it. Exposure is restricted on the **host** side instead, via `-p 127.0.0.1:5050:5050`, so the dashboard is not reachable from other machines on your network.

Build verification status

The Dockerfile and compose file have **not been built on this machine** — no container runtime was available in the development environment. What was verified: a directory containing exactly the files the Dockerfile copies was assembled, and `python run.py`, `python validate.py`, `python -m pytest -q` and importing `frontend.server:app` all succeed inside it. That confirms the `COPY` set is complete and the commands are correct; it does not substitute for an actual `docker build`.

Repository layout

kb.json	input: knowledge base passages
queries.json	input: user questions + answer constraints
candidate_answers.json	input: answers from each prompt variant
config.yaml	every threshold/weight, so behaviour changes without touching code
requirements.txt	pinned runtime dependencies
requirements-dev.txt	+ pytest, for running the test suite
evalharness/	the library
schemas.py	Pydantic models for every input and output file
loader.py	file loading + two-pass validation (structural, then referential integrity)
errors.py	Issue/InputValidationError – all problems in a fixture are collected and reported together
config.py	config.yaml loading + content hashing
hashing.py	file/text SHA-256 helpers (provenance, cache keys)
text.py	shared tokenizer, stemmer, stopword list
matching.py	the 3-rung phrase-matching ladder (exact → ordered-subsequence → stemmed-subsequence)
retrieval.py	deterministic BM25 index and search
checks.py	rule-based per-answer checks: retrieval_hit, must_include/must_not_claim, grounding_score
judge.py	the one controlled LLM review stage: prompt construction, structured-output validation, and the cache/live/stub backends
llm_log.py	llm_calls.jsonl read/append helpers
taxonomy.py	the six-tag failure taxonomy, derived from already-computed scores + judge output
aggregate.py	safety-gated aggregation and the recommendation.md renderer
review_report.py	the per-query explainability view renderer
manifest.py	run manifest: git SHA, config hash, input hashes, judge backend used, stage timings
run.py	orchestrates the full pipeline end to end (retrieval → scoring → LLM review → taxonomy → recommendation → explainability view → manifest) and writes all 8 output files
validate.py	checks a completed run's artifacts for presence, well-formedness, internal consistency, and recommendation reproducibility
frontend/	optional local dashboard (see above) — not part of the graded harness
server.py	FastAPI app (served by uvicorn): serves the UI, exposes the artifacts as JSON, runs

	run.py/validate.py/pytest as subprocesses, handles fixture upload (validated before it overwrites) and restore
static/index.html	page skeleton
static/app.js	rendering + fetch logic (no framework)
static/styles.css	hand-written CSS (no CDN, works offline)
tests/	
test_loader.py	validation behaviour, incl. adversarial fixtures
test_schemas.py	direct Pydantic model tests
test_matching.py	the phrase-matching ladder, incl. every near-miss case from the sample data and the one case correctly out of lexical scope
test_checks.py	grounding_score behavior: precision, quote bonus, numeric penalty, empty constraints
test_judge.py	schema validation + backend resolution (cache/live/stub), live backend mocked
test_taxonomy.py	each failure tag's derivation rule
test_aggregate.py	safety gate, tradeoff surfacing, k-variant support, no-clear-winner cases
test_determinism.py	repeated runs produce byte-identical retrieval + scoring output
test_pipeline_fixture_swap.py	the full pipeline (not just loading) run end-to-end against a synthetic 3-variant, empty-constraint fixture
test_frontend.py	the dashboard's endpoints via FastAPI's TestClient -- state payload shape, command wrappers, and that an invalid fixture upload is rejected without touching the real files. Skipped when the dashboard extras are absent
fixtures/broken/	hand-built fixtures, each violating exactly one rule, plus one fixture that must succeed (3 variants, unconventional names, empty constraint arrays) – proves fixture-swap resilience, not just the happy path

Generated artifacts (`retrieval.json` , `automated_scores.json` , `llm_review.json` , `llm_calls.jsonl` , `failure_taxonomy.json` , `recommendation.md` , `review_report.md` , `run_manifest.json`) are written to the repository root by `run.py` , matching the exact filenames the spec requires.

Artifacts reference

What each generated file actually contains, one record shape at a time.

retrieval.json — one record per query: `{query_id, retrieved: [{doc_id, score, title, text}]}`.
`retrieved` holds up to `config.retrieval.top_k` passages (fewer only if the knowledge base itself has fewer documents), highest BM25 score first, ties broken by ascending `doc_id` .

automated_scores.json — one record per (query_id, variant): {query_id, variant, retrieval_hit, must_include_pass, must_not_claim_pass, grounding_score, risk_flags, notes}. Every field here comes from deterministic code, never the LLM. risk_flags is ["none"] when nothing fired, otherwise any of retrieval_miss, missing_required_phrase, must_not_claim_violation, unsupported_numeric, low_grounding.

llm_review.json — one record per query: {query_id, winner, faithfulness: {variant: int}, clarity: {variant: int}, overclaim_flags: {variant: bool}, justification}. The three dicts are keyed by whatever variant names exist for that question — 2 or 20 — never a fixed pair of fields. winner is always one of those variant names.

llm_calls.jsonl — one line per backend attempt (not per successful call): {call_id, timestamp, backend, model, prompt_hash, parsed_ok, validation_errors, response_raw?, latency_ms?, usage?}. A single run.py invocation can append several lines (e.g. a failed live attempt followed by a successful stub fallback) — this is an append-only audit log, read it in full to see everything that was tried, not just the last line.

failure_taxonomy.json — one record per (query_id, variant): {query_id, variant, tags}, where tags is zero or more of the six fixed values named in the brief (unsupported_claim, missed_key_fact, policy_violation, retrieval_miss, overconfident_tone, irrelevant_answer).

recommendation.md — human-readable: selected variant, top reasons, tradeoffs (only rendered if any were found), a per-variant summary table (composite score, mean judge faithfulness/clarity, judge win count, failure tag counts), a per-query detail table, and a "known limitations of this run" section that names which judge backend actually served it.

review_report.md — the explainability view: for every query, the question, retrieved evidence with scores, every variant's answer text, which rule checks it failed, its failure tags, and the judge's winner for that question — everything needed to debug one query without cross-referencing four JSON files by hand.

run_manifest.json — provenance: generated_at, git_commit (best-effort, null outside a git checkout), config_hash, input_hashes (SHA-256 of each of the 3 input files), python_version, judge.{model, backend_order, backend_used}, counts, and stage_timings_seconds.

How the recommendation is computed

The full rule, in order, exactly as implemented in evalharness/aggregate.py :

1. **Safety gate (a veto, not a penalty).** For every variant, walk every query it answered. If must_not_claim_pass is false on a query whose risk_level meets or exceeds config.gates.disqualify_must_not_claim_at_severity (default: high and above), that variant is **disqualified outright** — no composite score, however high, can buy it back. This is deliberate: a variant that is clearer, better-grounded, and wins every other query but tells one customer their password can be emailed to them must never be promotable.
2. **Composite score, for every variant (including disqualified ones).** Per query, a weighted blend of retrieval_hit, must_include_pass, must_not_claim_pass, grounding_score, and the judge's faithfulness / clarity / winner-vote (weights in config.aggregation.weights), averaged across all queries that variant answered. Disqualified variants still get a composite — it's what lets the report show *how good it otherwise was*, which is exactly what the tradeoff-surfacing step below needs.
3. **Pick among the survivors.** - Zero variants pass the gate → **no promotion recommended**, reasons listed. - Exactly one passes → **it is promoted**, no further comparison needed. - More than one passes → the highest composite score wins, but only if its margin over the runner-up is at least

`config.aggregation.min_margin_for_promotion` (default `0.02`). Below that margin, the evidence is treated as **too thin to prefer one over the other** — the harness explicitly recommends no promotion rather than picking a coin-flip winner and dressing it up as a finding.

4. **Surface the tradeoff, automatically.** If any disqualified variant's mean judge-rated clarity is higher than the promoted (or best surviving) variant's, that is called out by name in `recommendation.md` 's "Tradeoffs" section — the literal case the brief asks for: "explain tradeoffs if one variant is clearer but less safe."

Every one of these four steps reads only already-written artifacts (`automated_scores.json` , `llm_review.json` , `failure_taxonomy.json`) plus `config.yaml` — nothing here re-touches raw answer text or calls the judge again, which is what makes `validate.py` 's reproducibility check (below) meaningful rather than circular.

LLM review stage: enabling live judging

The judge stage resolves through three backends, in this order (see `judge.backend_order` in `config.yaml`):

1. **cache** — replays a previously logged live call from `llm_calls.jsonl` by exact content hash (provider + model + params + prompt + rubric version). Zero cost, fully offline, byte-identical output.
2. **live** — a real API call to the configured provider, temperature 0, structured output enforced via that provider's forced tool call, then re-validated in code before use.
3. **stub** — a deterministic, rule-derived judgment (no network, no model) used when no key is configured or the live call fails. The pipeline always completes; `llm_calls.jsonl` , `run_manifest.json` , `recommendation.md` , and the dashboard all record which backend actually served each run, so a stub result is never silently mistaken for a real model judgment.

Two providers are supported, selected by `judge.provider` in `config.yaml` (`openai` or `anthropic`). Adding another is one function plus one registry entry in `evalharness/judge.py` — nothing else changes.

```
# OpenAI (the default in config.yaml)
pip install -r requirements-llm.txt
export OPENAI_API_KEY=...
python run.py

# Anthropic: set judge.provider: anthropic and judge.model in config.yaml
pip install anthropic
export ANTHROPIC_API_KEY=...
python run.py
```

The first run makes one real API call covering every query/answer pair and appends it to `llm_calls.jsonl`. Every subsequent run replays that exact call from cache — free, offline, deterministic — until the inputs, config, prompt, provider, or model change, at which point the content hash changes and a fresh live call is made.

Both providers use **forced function/tool calling** rather than a strict JSON-schema response format. The review payload keys its per-variant score maps by variant *name* — which is data, not schema — and strict schema modes disallow the open-ended `additionalProperties` that requires.

Verified against the live OpenAI API

The live path has been exercised end to end against `openai/gpt-4o-mini` (1,353 input / 419 output tokens for the full 4-query batch). Two things that verified:

- **Cache replay is real.** A second run with `OPENAI_API_KEY` unset resolved to the `cache` backend and produced byte-identical `llm_review.json` output. That is the whole "replayable pipeline with a genuine LLM stage" claim, demonstrated rather than asserted.
- **The judge closed the gap the lexical layer cannot.** Q4's banned claim ("yes, after verification") shares almost no tokens with `prompt_b`'s "You may be able to withdraw after your demo account is upgraded and verified", so the deterministic matcher correctly does not fire on it. The live judge caught it anyway — flagging `overclaim_flags: {prompt_b: true}` with the reasoning "introduces uncertainty by suggesting that withdrawals may be possible after upgrading, which is not supported by the evidence." That is precisely the division of labour this harness is built around: cheap deterministic checks handle lexical drift, and the one LLM stage is reserved for the semantic judgment code genuinely cannot make.

`llm_calls.jsonl` in this repo therefore contains a real `live` entry plus its `cache` replay. **No API key is stored in this repository** — keys are read from the environment only, and never written to any artifact, config file, or log.

Validation

`python validate.py` (or `make validate`) checks a completed run without needing to re-run the pipeline:

1. every required artifact exists and is syntactically valid JSON/JSONL (Markdown files are checked for non-empty content)
2. every artifact's records conform to their Pydantic schema
3. every `(query_id, variant)` pair present in `candidate_answers.json` has a corresponding `automated_scores.json` record
4. `retrieval.json` contains exactly `config.retrieval.top_k` passages per query (or fewer only if the knowledge base itself has fewer documents than that)
5. every `llm_review.json` record uses only real variant names and in-range scores — this reuses the exact same `validate_reviews()` function the judge stage applies to a live API response, so there is one source of truth for "a valid review," not two implementations that could drift apart
6. `recommendation.md` is **recomputed from the stored artifacts** (`automated_scores.json`, `llm_review.json`, `failure_taxonomy.json`, plus the judge backend recorded in `run_manifest.json`) and diffed byte-for-byte against what's on disk — if a fixture or config changed since the last `run.py`, this fails loudly rather than silently serving a stale recommendation

Exit code 0 means every check passed; exit code 1 means at least one failed, with every failure printed (not just the first). This was verified by deliberately corrupting each of a stale `recommendation.md`, a truncated `automated_scores.json`, and an invalid `winner` in `llm_review.json` — each was caught with a specific, correct error message, then the artifacts were restored.

Design principles this repo follows

- **Schema, never data.** Nothing in `evalharness/` references a specific query id, KB doc id, phrase, or variant name. Variant names (`prompt_a`, `prompt_b`, ...) are read from `candidate_answers.json` as dictionary keys; the pipeline works unmodified for 1, 2, or N variants under any names. This is asserted by `tests/test_loader.py::TestFixtureSwapResilience`.
- **Fail fast, but report everything at once.** A malformed or inconsistent fixture raises `InputValidationError` before any pipeline stage runs, and that error lists every issue found — not just the first — so a swapped-in fixture with several problems doesn't need several rounds of fixing-and-rerunning to see them all.

- **Referential integrity is checked, not assumed.** The loader verifies that `expected_doc_ids` in `queries.json` actually resolve into `kb.json`, that every answer record references a real query, and that every query has a consistent set of answer variants. These are exactly the kind of cross-file bugs a hand-edited fixture can introduce silently.
- **Config over code.** Every tunable — BM25 parameters, matching tolerance, grounding score weights, risk severities, gate thresholds, aggregation weights, judge settings — lives in `config.yaml`. Its content hash is recorded in `run_manifest.json` (alongside the git commit and every input file's SHA-256) so a recommendation's exact settings are always traceable after the fact, not just at the moment it was made.

Design tradeoffs and why

Concrete decisions made during the build, and the reasoning behind each — distinct from "Known limitations" below, which lists the *consequences* of these choices rather than the choices themselves.

- **BM25 over embeddings, for retrieval.** Deterministic, fully local, no model download, no network call — and the brief explicitly disallows external calls for retrieval. The tradeoff: BM25 is weak on a query that shares few tokens with its answer passage (pure paraphrase). No embedding-based fallback is included; see "What I'd add next."
- **A 3-rung lexical matching ladder, not pure substring or pure semantic matching.** Free, instant, and fully auditable — the rung that matched is recorded in every check's notes. It resolves the majority of lexical drift seen in the actual sample data (an inserted word, a plural/verb form, a reworded copula) without a model call. The one case that is genuinely semantic-only (no shared tokens at all) is deliberately left unmatched by design, not patched around — that gap is exactly what the LLM judge stage exists to close.
- **One batched LLM call across every query/answer pair, not one call per answer.** This is what the brief asks for, and it has real consequences: latency and cost are bounded and predictable, and the pipeline's only non-deterministic seam is a single, cacheable call rather than scattered throughout. The cost is atomicity — if any part of the batch response fails validation, the *entire* batch is rejected and retried/falls back, rather than salvaging the queries that parsed correctly. Chosen deliberately: partial acceptance would mean a recommendation could silently mix judged and un-judged queries with no visible marker of which was which.
- **Structured output enforced via forced tool use, then re-validated in code anyway.** A JSON-schema tool definition can require "an object with these keys" but cannot express "an integer between 1 and 5" or "these dict keys must exactly equal this question's variant set, whatever that set happens to be." Code-side validation is not redundant with the API's structured-output guarantee — it checks things the schema mechanically cannot.
- **The safety gate is a hard veto, not a weighted term in the composite score.** A weighted-penalty design was considered and rejected: with large enough weights on faithfulness/clarity, a strong-enough advantage there could always mathematically outvote a real safety violation for some input. A veto has no such failure mode — no composite score, however high, can un-disqualify a variant that told a customer their password could be emailed to them.
- **Backend order `cache -> live -> stub`, and a stub that says plainly it is not a real judgment.** A live-only design would fail closed for any evaluator without an API key, on a task that explicitly says "if no API key is available, your pipeline should still run ... or provide a stubbed path." A fully mocked judge would dodge the requirement to build a real integration at all. This design does both: a genuine Anthropic tool-use integration exists and is unit-tested, and the pipeline still completes with zero configuration.
- **A single winner-takes-one recommendation, with an explicit "no promotion" outcome — not a leaderboard or a forced pick.** The brief asks for a recommendation, not a report a human still has to interpret. Refusing to promote when the margin is thin is a deliberate stance against manufacturing false confidence from noisy per-query signals.

- **Pydantic `extra="forbid"` on every output model, `extra="allow"` on every input model.** Outputs are this harness's own contract and should be exact. Inputs may legitimately carry fields an evaluator's fixture uses for its own bookkeeping that this harness doesn't need — rejecting those outright would be needless brittleness in the direction that matters least.
- **Every threshold lives in `config.yaml`, none in code.** BM25 parameters, the matching gap budget, grounding weights, risk severities, the disqualifying severity, aggregation weights, the promotion margin, judge settings — all of it. This is what makes "a sensible tradeoff" auditable after the fact rather than an implicit assumption buried in a function: the config's content hash is in every run's manifest, so the exact settings behind any past recommendation can always be recovered.

Known limitations

- The rule-based `must_not_claim` / `must_include_any` matcher is lexical (exact → ordered-subsequence → stemmed-subsequence), not negation-aware. A hedged or explicitly negated mention of a banned phrase can register as a match; this is intentional (high recall over high precision at the deterministic layer) and is exactly the ambiguity the LLM judge stage is positioned to resolve.
- `grounding_score` is a token-overlap-plus-heuristics proxy for faithfulness, not a semantic entailment check. It cannot catch a claim built entirely from words that individually appear in the evidence but combine into something the evidence never said.
- The stub judge backend derives `winner` / `faithfulness` / `overclaim_flags` from the same deterministic signals already in `automated_scores.json`; it adds no independent judgment. `clarity` under the stub is a constant placeholder, not a real assessment. This is why the backend actually used is surfaced everywhere the results are — a stub run is a valid fallback, not an equivalent substitute for the live judge.
- BM25 retrieval, like any lexical retriever, is weakest on a query that shares few tokens with its answer passage (pure paraphrase). This harness has no embedding-based fallback by design (external calls for retrieval are disallowed by the brief).
- The composite score's weights (`config.yaml: aggregation.weights`) are a stated, documented modeling choice, not a discovered optimum. They are designed to be easy to audit and change, not to be treated as the one correct weighting — this is also called out live in every generated `recommendation.md`.
- With only 4 queries in the shipped sample fixture, any composite margin computed against it has limited statistical power; `recommendation.md` says so explicitly. A larger fixture would give the margin more meaning, but the harness itself places no lower bound on query count.
- `run_manifest.json`'s `git_commit` field is best-effort: outside a git checkout, or if the `git` binary is unavailable, it is recorded as `null` rather than failing the run.

What I'd add next

Roughly in the order I'd tackle them:

1. **A key-gated integration test for the live judge.** The live path has now been exercised for real against `openai/gpt-4o-mini` (see [Verified against the live OpenAI API](#)), and everything downstream of "a response came back" is unit-tested against a mocked call. What's missing is *continuous* coverage: a test that runs only when an API key is present in the environment and is skipped otherwise, so the real request path stays covered on any machine that has a key rather than being verified once by hand. The Anthropic backend in particular is still only mock-tested.
2. **A negation-aware check to close the lexical ladder's most-cited gap.** The matcher can register a hedged or explicitly negated mention of a banned phrase as a violation. A small local NLI model or a negation-scope heuristic

over the matched span would tighten this without giving up the ladder's speed and auditability for the common case.

3. **Statistical rigor on the promotion margin.** The current check is a single fixed threshold (`min_margin_for_promotion`). A paired bootstrap confidence interval over the per-query composite differences would let "no promotion" be backed by a stated confidence level instead of an arbitrary cutoff — much more defensible at a larger query count than the 4 in the shipped fixture.
4. **An embedding-based retrieval fallback**, still fully local (e.g. a small sentence-transformer with no network call at inference time), for the subset of queries where BM25's token overlap is weak. Alongside it: report `recall@k / MRR` as a retrieval-quality health diagnostic in its own right, not just as an input to `retrieval_hit`.
5. **CI.** A GitHub Actions workflow running `pytest`, `python run.py`, and `python validate.py` on every push, so "runs from a clean checkout" is continuously verified rather than something I confirmed manually once before handing it over.
6. **A cost/latency rollup across runs.** `llm_calls.jsonl` already logs token usage and latency per call; nothing yet aggregates it into a \$-and-seconds summary across many runs. That's a small addition on top of data the harness already collects.
7. **A pairwise judge mode as a config-toggleable alternative** to the current pointwise-with-winner design, with order-swap bias correction — for teams that want the extra sensitivity a head-to-head comparison gives and are willing to pay for the added complexity of correcting for position bias.
8. **A multi-model judge ensemble** (two different judge models voting) behind the same batching/caching machinery, to reduce how much the one genuinely non-deterministic stage depends on a single model's idiosyncrasies.

A note on the sample answer data

The reference screenshots providing `candidate_answers.json` initially truncated two answer strings mid-sentence (`prompt_a` for Q1 and Q2). They were completed by hand for the first commit and later replaced with the full, authoritative text once it was provided. `candidate_answers.json` now reflects the exact wording as given, with no edits. Since the harness is schema-driven and swap-tolerant, the precise wording of the sample answers was never load-bearing for the pipeline itself — this note exists purely for a transparent history of the data.